

Core-V Wally (CVW): Interrupt Handling in C

A Practical Programming Guide for Bare-Metal RISC-V

1. Executive Overview

In **Core-V Wally (CVW)**, C programs execute in Machine Mode on a bare-metal RISC-V processor architecture. Interrupt processing is structured around the RISC-V privilege specification, utilizing low-level assembly startup code for trap vectoring and register context preservation, while exposing a flexible C interface for custom application handlers.

Stage	Component & Location	Description
1. Vectoring	mtvec CSR in crt.S	Startup code sets Machine Trap Vector Base Address to trap_entry.
2. Context Save	trap_entry in crt.S	Saves CPU registers (x1-x31) onto stack, reads mcause, mepc, and sp.
3. C Callback	handle_trap() in syscalls.c	Dispatches to the weak handle_trap() C function, which can be overridden in user C code.
4. Restore & Return	mret in crt.S	Restores mepc, pops registers from stack, and executes mret to resume.

2. Step-by-Step C Implementation Guide

Step 1: Override handle_trap() in your C code

The standard runtime declares handle_trap as weak. Define your custom version to inspect the cause:

```
uintptr_t handle_trap(uintptr_t cause, uintptr_t epc, uintptr_t regs[32]) {
    // Check MSB for interrupt flag (bit 63 for RV64, bit 31 for RV32)
    uintptr_t is_interrupt = cause & (1ULL << (__riscv_xlen - 1));
    uintptr_t code = cause & 0xFFF;

    if (is_interrupt) {
        if (code == 7) { /* Handle Machine Timer Interrupt */ }
        if (code == 11) { /* Handle Machine External Interrupt (PLIC) */ }
        return epc; // Resume execution at epc
    }
    return epc + 4; // Exception handling (advance instruction)
}
```

Step 2: Configure Peripherals (CLINT / PLIC)

- **Timer Interrupts (CLINT):** Write target tick count to MTIMECMP (0x02004000).
- **External Interrupts (PLIC):** Set source priority (0x0C000000 + 4*ID), enable ID in context (0x0C02000), and set threshold to 0.

Step 3: Enable Interrupts via RISC-V CSRs

Use macros defined in `examples/C/common/encoding.h`:

```
// Enable specific interrupt in mie CSR
set_csr(mie, MIE_MTIE); // Machine Timer Interrupt Enable (bit 7)
set_csr(mie, MIE_MEIE); // Machine External Interrupt Enable (bit 11)

// Enable global interrupts in mstatus CSR
set_csr(mstatus, MSTATUS_MIE); // Machine Global Interrupt Enable (bit 3)
```

3. Complete Code Example: Machine Timer Interrupt

The following standalone C application initializes a periodic timer interrupt on Core-V Wally:

```
#include <stdint.h>
#include "encoding.h" // Include from examples/C/common/encoding.h

#define MTIME      (*(volatile uint64_t*)0x0200BFF8)
#define MTIMECMP  (*(volatile uint64_t*)0x02004000)

volatile uint32_t timer_count = 0;

// Overriding the weak trap handler from syscalls.c
uintptr_t handle_trap(uintptr_t cause, uintptr_t epc, uintptr_t regs[32]) {
    uintptr_t is_interrupt = cause & (1ULL << (__riscv_xlen - 1));
    uintptr_t code = cause & 0xFFF;

    if (is_interrupt && code == 7) { // Machine Timer Interrupt
        timer_count++;
        MTIMECMP = MTIME + 10000;    // Acknowledge interrupt & set next timer threshold
        return epc;                  // Return to interrupted instruction
    }
    return epc + 4;
}

void init_timer(uint64_t ticks) {
    MTIMECMP = MTIME + ticks;
    set_csr(mie, MIE_MTIE);          // Enable timer interrupts in mie
    set_csr(mstatus, MSTATUS_MIE);   // Enable global interrupts in mstatus
}

int main(void) {
    init_timer(10000);
    while (timer_count < 5) {
        // Main loop continues execution while interrupts increment timer_count
    }
    return 0;
}
```

4. External Hardware Interrupt (PLIC / UART Example)

For external device interrupts routed through the PLIC (such as UART source ID 10 on Wally):

```
#define PLIC_PRIORITY(id) (*(volatile uint32_t*)(0x0C000000 + 4 * (id)))
#define PLIC_ENABLE      (*(volatile uint32_t*)(0x0C002000))
#define PLIC_THRESHOLD    (*(volatile uint32_t*)(0x0C200000))
#define PLIC_CLAIM        (*(volatile uint32_t*)(0x0C200004))

// Inside handle_trap for cause == 11 (Machine External Interrupt):
uint32_t claim_id = PLIC_CLAIM;    // Read active interrupt source ID
if (claim_id == 10) {               // UART source ID on Core-V Wally
    // Read/clear peripheral data or register
}
PLIC_CLAIM = claim_id;              // Write back claimed ID to signal completion
```

5. Key Reference Files in this Repository

File Path	Description
examples/C/common/crt.S	C startup code establishing mtvec and trap_entry assembly frame.
examples/C/common/syscalls.c	Contains default weak handle_trap() implementation and UART output routines.
examples/C/common/encoding.h	RISC-V CSR definitions and helper macros (set_csr, read_csr).
examples/asm/trap/trap.S	Assembly example demonstrating timer interrupt and ecall handling.
examples/asm/mei/mei.S	Assembly example for Machine External Interrupts via PLIC and UART.